PROJECT JOURNEYS · PRODUCTION ML & AGENTIC AI

Two Platforms, One Discipline: Reliable ML & Multi-Agent AI in Regulated Finance

01 Lending Decision Platform
Google Cloud · Vertex AI · GKE · Argo · Gemini + Claude

02 Fraud Detection Platform
AWS · SageMaker · EKS · EMR/Spark · Airflow · Claude

Staff ML Engineer — interview presentation

HOW I'LL WALK YOU THROUGH THIS

What this presentation covers

01

The journey

Discovery → design → development → delivery, for each platform

02

My contribution

What I personally owned vs. what the team delivered, mapped to the role

03

Architecture

Full solution diagrams: data, training, serving, agents + RAG

04

Decisions & trade-offs

Why GCP vs AWS tooling, managed vs self-managed, model & LLM choices

05

Challenges

Regulatory explainability, scale, drift, false positives — and the fixes

06

Engineering practice

Release management, ML monitoring, and MLflow / TensorFlow / PyTorch

THE TWO PLATFORMS AT A GLANCE

Same playbook, two clouds, two risk problems

01 LENDING DECISION · Google Cloud

Goal Automate & de-risk credit decisioning; consistent, explainable, compliant lending

Core ML Calibrated gradient-boosted lending-decision model + reason codes

Orchestration Argo Workflows (training DAGs) on GKE

Serving Vertex AI endpoints + agent services on GKE

LLMs Gemini + Claude (task-routed) with RAG over lending policy

Experiments Databricks + MLflow

02 FRAUD DETECTION · AWS

Goal Catch fraud in near-real-time at scale; minimise loss & customer friction

Core ML Hybrid supervised + unsupervised scoring on extreme class imbalance

Orchestration Airflow (training DAGs); EMR/Spark for feature compute

Serving SageMaker endpoints + low-latency scoring on EKS

LLMs Claude for investigation agents + RAG over case/AML knowledge

Experiments Databricks + MLflow

PROJECT 01

Lending Decision Platform

Google Cloud · agentic credit decisioning with explainability and compliance built in

LENDING · THE PROBLEM WE SET OUT TO SOLVE

Faster, consistent, defensible credit decisions

Business context

Manual underwriting was slow, inconsistent across officers, and hard to audit. The bank wanted automated decisioning that was fast, consistent, and — critically — explainable and compliant with responsible-lending obligations.

The shape of the solution

A calibrated ML decision model at the core, wrapped in four specialised AI agents that intake the application, run KYC/AML, analyse finances, and audit for security — all grounded in current lending policy via RAG, with a human in the loop for high-value / low-confidence cases.

≤ minutes

decision turnaround vs. hours of manual review

Reason codes

on every decision — adverse-action ready & auditable

HITL gate

humans review high-value & low-confidence cases

How it unfolded: discovery to delivery

1

Discovery & framing

- Workshops with Lending, Risk & Compliance
- Mapped decision policy & data sources
- Defined success metrics + guardrails
- Feasibility spike on GCP

2

Design & architecture

- Chose GCP-native stack
- Agent decomposition (4 agents)
- Argo training DAGs + feature store
- Explainability & audit by design

3

Development

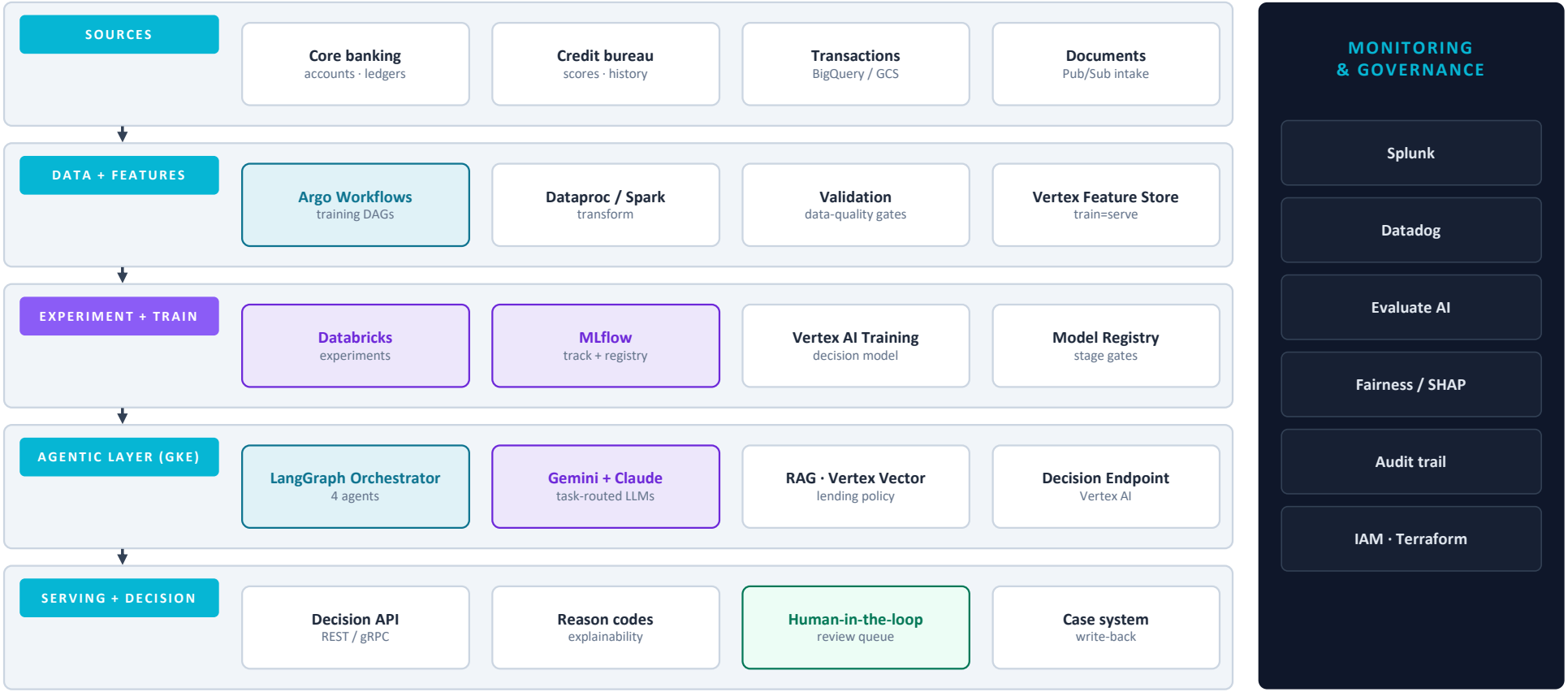
- Data pipelines + feature store
- Decision model + calibration
- LangGraph agents + RAG over policy
- CI/CD via Terraform; eval harness

4

Delivery & rollout

- Shadow → canary → progressive
- Human-in-the-loop integration
- Model-risk review sign-off
- Monitoring + reason-code audit trail

Solution architecture — end to end



How model training works

Orchestrated by Argo Workflows (GKE-native) — every run idempotent, versioned, reproducible

Retraining is triggered, not scheduled by habit. Drift monitors or accumulated label corrections fire a run; the gate blocks promotion unless metric, fairness and calibration checks pass.

Point-in-time features

One definition serves training and the live decision endpoint — no skew, no leakage.

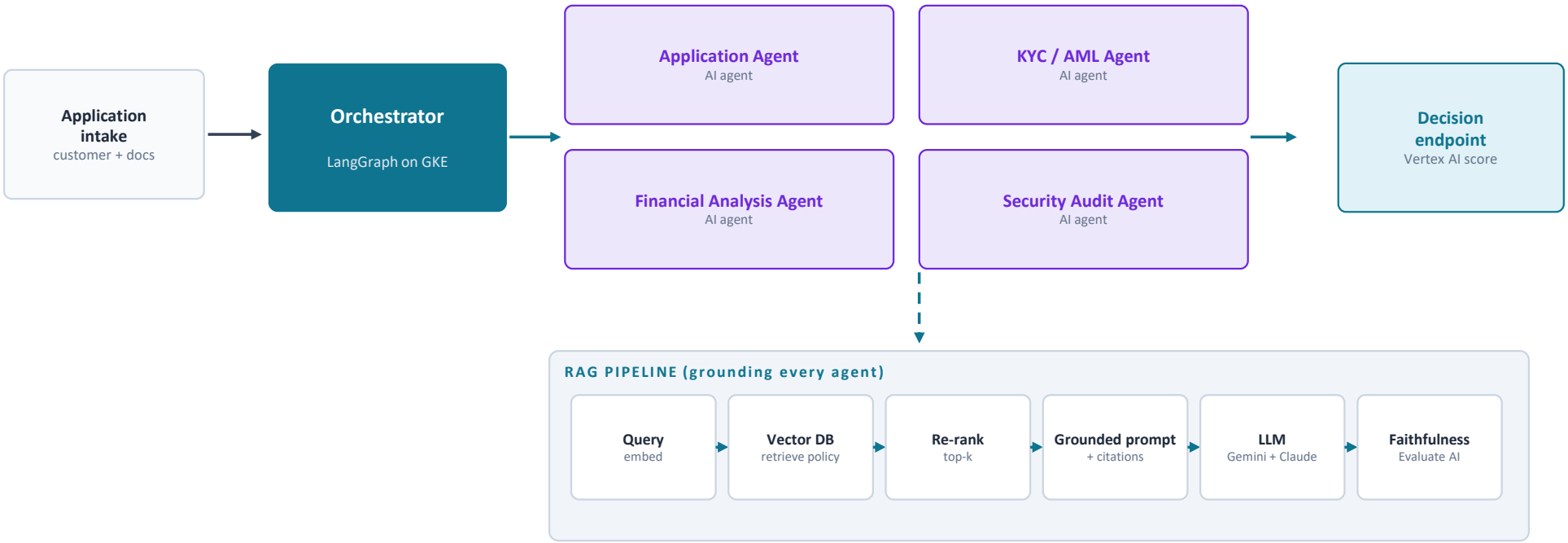
Fairness as a gate

Sliced error + adverse-impact checks run in CI, not as a post-hoc audit.

Calibrated probabilities

Decisions ride on probabilities, so isotonic/Platt calibration is mandatory.

AI agents + RAG pipeline



Agents call the ML scoring endpoint + data tools, then return a grounded, cited recommendation → reason codes → human-in-the-loop for the edge cases.

01 · LENDING DECISION PLATFORM

What I owned vs. what the team delivered

What I personally owned · Staff scope

- Set the technical direction & overall architecture across the platform
- Designed the Argo training pipeline + Vertex Feature Store (train=serve)
- Designed the LangGraph agent orchestration & RAG grounding strategy
- Established the MLOps standards: MLflow registry, CI/CD, release gates
- Owned the model evaluation, fairness & calibration approach
- Mentored 5 engineers; ran the design reviews that set the bar

What the team delivered

- Built source-system ingestion connectors & data contracts
- Implemented individual agent logic against the framework I defined
- Implemented model training code from the pipeline templates
- Ran GKE platform / SRE and on-call operations
- Credit analysts validated decision quality & reason codes
- QA & compliance executed UAT and sign-off

Maps to the role: System design at scale · Setting standards · Mentoring · MLflow/TF/PyTorch · LLM features

01 · LENDING DECISION PLATFORM

Client & stakeholder intersection

Lending & Credit Risk

Co-owned the decision policy; I translated their rules into model logic, thresholds & reason codes.

Compliance & Legal

Responsible-lending obligations → explainability, fairness testing & audit requirements I designed to.

Model Risk Review

Presented model methodology & validation for formal sign-off before each promotion.

Product & Underwriters

Designed the human-in-the-loop review flow with the people who'd actually use it.

Data Governance

Agreed data lineage, access controls & retention for regulated financial data.

Cloud / Google

Architecture reviews on Vertex AI & GKE; negotiated managed-service trade-offs.

Technology decisions & trade-offs

Decision	What I chose	Trade-off I accepted
Decision model class	**Calibrated gradient-boosted trees**	Less 'cutting-edge' than deep nets — but explainable, fast, regulator-friendly on tabular data
Serving	**Vertex AI managed endpoints**	Some GCP lock-in, in exchange for built-in monitoring & faster, safer delivery
Training orchestration	**Argo Workflows on GKE**	More ops than Vertex Pipelines, bought portability & k8s-native flexibility
Policy knowledge	**RAG over fine-tuning**	More moving parts, but fresh, citable policy — essential for compliance defensibility
LLMs	**Gemini + Claude, task-routed**	Multi-vendor complexity, in exchange for best-fit quality/cost & no single-vendor lock-in

01 · LENDING DECISION PLATFORM

Challenges & how I worked through them

⚠ Decisions had to be explainable & fair

Interpretable model + SHAP reason codes + adverse-action notices; fairness tested across protected attributes with a full audit trail.

⚠ LLM hallucination in financial analysis

Grounded every agent via RAG, verified any numeric claim against source data, and gated outputs through the Evaluate AI faithfulness harness.

⚠ Multi-agent latency vs thoroughness

Ran agents in parallel, cached retrievals & embeddings, and held a strict SLA budget with graceful degradation to a simpler path.

⚠ Disparate, messy source data

Data-quality gates at every boundary + a feature store as the single source of truth, so bad data never reached training or serving.

PROJECT 02

Fraud Detection Platform

AWS · real-time detection at scale on extreme class imbalance, with agentic investigation

FRAUD · THE PROBLEM WE SET OUT TO SOLVE

Catch more fraud, frustrate fewer customers

Business context

Fraud losses were rising and the existing rules engine was both leaky and noisy — too much fraud got through, while too many genuine customers were wrongly blocked. The mandate: score transactions in near-real-time, at the scale of millions, and cut losses without adding customer friction.

The shape of the solution

A hybrid ML scorer (supervised + unsupervised) on a streaming pipeline, with four investigation agents that triage alerts, run AML checks, analyse finances and audit security — surfacing a grounded case summary to fraud analysts.

~0.1%

fraud base rate — extreme imbalance drove every metric choice

Near-real-time

sub-second scoring on streaming transactions at scale

Analyst-in-loop

agents draft the case; humans decide the edge

How it unfolded: discovery to delivery

1

Discovery & framing

- Workshops with Fraud Ops & Financial Crime
- Quantified loss & false-positive cost
- Set metrics: recall@precision, $ saved
- Reviewed adversarial threat model

2

Design & architecture

- Chose AWS-native stack
- Streaming + batch feature paths
- Airflow training DAGs; EMR/Spark
- Same 4-agent investigation pattern

3

Development

- Streaming pipeline + online features
- Hybrid supervised/unsupervised scorer
- Claude investigation agents + RAG
- SageMaker pipelines + MLflow

4

Delivery & rollout

- Shadow scoring against the rules engine
- Champion–challenger, then ramp
- Analyst feedback loop wired in
- Drift monitoring + auto-retrain

Solution architecture — end to end

How model training works

Orchestrated by Airflow + SageMaker Pipelines — every run idempotent, versioned, reproducible

Retraining is triggered, not scheduled by habit. Drift detectors and the analyst feedback loop trigger retraining; champion–challenger validates a new model in shadow before it ever takes traffic.

Imbalance-aware

PR-AUC & recall-at-fixed-precision, resampling on the train fold only — never raw accuracy.

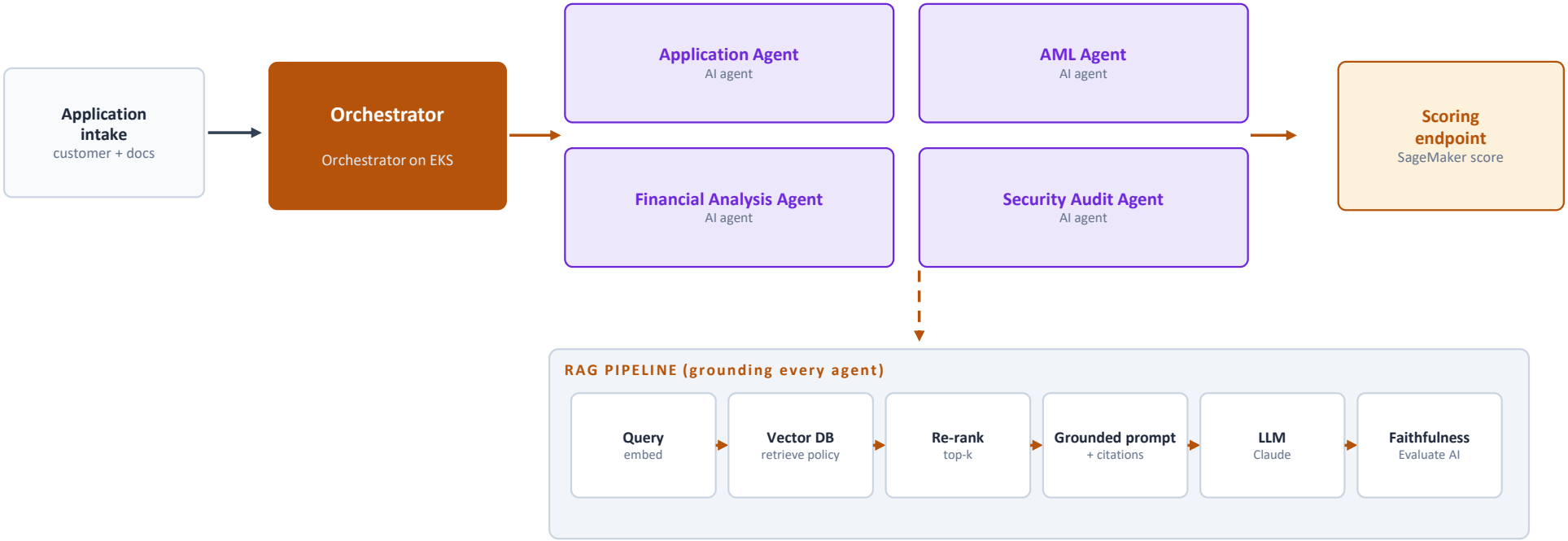
Distributed at scale

EMR/Spark computes billions of behavioural & velocity features; the online store serves them sub-second.

Delayed labels

Fraud is confirmed late, so the pipeline handles label lag & weak labels explicitly.

AI agents + RAG pipeline



Agents call the ML scoring endpoint + data tools, then return a grounded, cited recommendation → reason codes → human-in-the-loop for the edge cases.

02 · FRAUD DETECTION PLATFORM

What I owned vs. what the team delivered

What I personally owned · Staff scope

- Owned the end-to-end architecture & the system-design-at-scale decisions
- Designed the EMR/Spark feature pipeline + online feature store
- Set the imbalance-aware evaluation & threshold-tuning methodology
- Designed champion–challenger release & the analyst feedback loop
- Reused & hardened the agent + RAG framework across both platforms
- Mentored engineers; drove the cross-team standards that unified both clouds

What the team delivered

- Built streaming ingestion & Spark feature jobs to spec
- Implemented model training & the agent tools
- Ran EKS platform, autoscaling & on-call
- Fraud analysts labelled cases & tuned review thresholds with us
- SecOps integrated detection with incident response
- QA executed load & latency testing at scale

Maps to the role: System design at scale · Spark / distributed · Setting standards · Mentoring · Real-time serving

02 · FRAUD DETECTION PLATFORM

Client & stakeholder intersection

Fraud Operations

Co-designed the alert queue & case summaries with the analysts who work them daily.

Financial Crime / AML

Aligned detection & agent reasoning with AML typologies and reporting obligations.

Customer Experience

Tuned thresholds against false-positive friction — protecting genuine customers from blocks.

Risk & Compliance

Presented model validation & monitoring evidence for governance sign-off.

Data Platform / SRE

Agreed streaming SLAs, autoscaling & cost guardrails for real-time scoring.

SecOps

Wired detections into incident response & the security audit agent's signals.

02 · FRAUD DETECTION PLATFORM

Technology decisions & trade-offs

Decision	What I chose	Trade-off I accepted
Model approach	**Hybrid supervised + unsupervised**	More complexity, but catches both known fraud and novel/zero-day patterns
Feature compute	**EMR/Spark over AWS Glue**	More cluster ops, bought heavy distributed throughput & spot-instance cost savings
Training & serving	**SageMaker managed**	Some lock-in, in exchange for managed pipelines, registry & autoscaled endpoints
Orchestration	**Airflow over Step Functions**	Self-managed, but richer DAGs & portability across our stack
Decision action	**Step-up auth, not just block**	More flows to build, but far less customer friction than hard blocks

02 · FRAUD DETECTION PLATFORM

Challenges & how I worked through them

⚠ Extreme class imbalance (~0.1%)

Anchored on PR-AUC & recall-at-fixed-precision tuned to the cost matrix; resampled only inside the training fold; calibrated scores.

⚠ Sub-second latency at scale

Precomputed heavy features in the online store, computed only light features live, cached aggressively, autoscaled on EKS with graceful fallback.

⚠ Adversaries & concept drift

Drift monitors + champion–challenger + frequent triggered retraining so the model keeps pace with evolving fraud.

⚠ False positives hurt real customers

Tuned thresholds to analyst capacity, added step-up auth, and fed analyst decisions back as labels to sharpen precision over time.

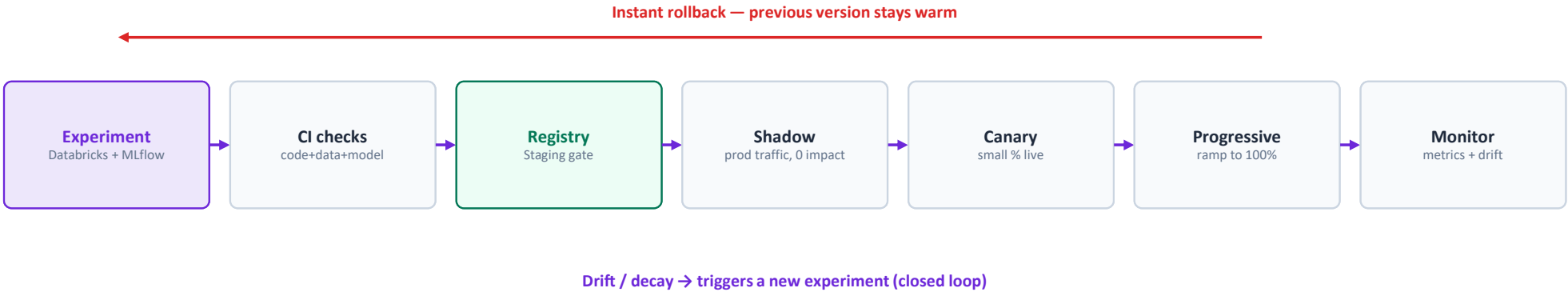
PROJECT

Engineering Practice

The cross-cutting discipline behind both platforms — release, monitoring, and the ML framework strategy

FAST-PACED DEVELOPMENT → PRODUCTION, SAFELY

Release management without compromising production



Every gate is a quality bar

Promotion is blocked unless metric, fairness, calibration & guardrail checks all pass.

Progressive, never big-bang

Shadow → canary → ramp means a bad model is caught on a sliver of traffic, not all of it.

Versioned & traceable

Each prediction maps to a model version, data & feature snapshot — fully auditable & reversible.

Champion–challenger

New models prove themselves against the incumbent in shadow before taking real traffic.

KNOWING THE MODEL STILL WORKS IN PRODUCTION

Monitoring & measuring ML metrics and accuracy

Operational health

Datadog

Latency p50/p99, throughput, error rate, saturation, cost per 1k predictions

Data & feature drift

PSI / KS monitors

Input distribution shift, prediction-distribution shift, feature freshness & null spikes

Model performance

Sliced metrics

Lending: AUC, calibration, fairness by segment · Fraud: PR-AUC, recall@precision, $ saved

LLM / agent quality

Evaluate AI

Faithfulness, citation validity, refusal/leak checks on every agent output

Splunk aggregates logs & fires alerts across all four layers. Alert thresholds are tied to business impact, not arbitrary statistics — minor drift logs & watches, sustained decay triggers retraining, a sharp break rolls back. Ground-truth performance is tracked as labels arrive (immediately for lending, with deliberate lag handling for fraud), so we measure real accuracy, not just proxies.

MODEL DEVELOPMENT STRATEGY

MLflow, TensorFlow & PyTorch — how they fit together

MLflow — the connective tissue. One tracking + registry layer for both frameworks and both clouds: every run's params, data & feature versions, metrics and artifacts are logged; the registry governs Staging→Production promotion with the release gates. Framework-agnostic by design.

PyTorch · research & iteration

- Fast, flexible experimentation — the default for research & rapid iteration
- Where we explore new architectures and fine-tune transformer / LLM components
- Dynamic graphs make debugging & custom logic straightforward
- Served via TorchServe, or exported to ONNX where a common runtime helps

TensorFlow · production serving

- Mature serving & lifecycle ecosystem (TF Serving, TFX) for stable production
- Preferred where we need a hardened, high-throughput serving path
- Strong fit for managed endpoints on Vertex AI / SageMaker
- Graph optimisation & quantisation for low-latency, low-cost inference

The strategy in one line: *prototype freely in PyTorch, standardise artifacts & metrics through MLflow, then serve on whichever runtime meets the latency, cost & ecosystem needs — gradient-boosted models still win on tabular decisioning, and that's a feature, not a compromise.*

WHAT THESE TWO JOURNEYS SHOW

The through-line

Production ML at scale

Both platforms serve real decisions to real users — feature stores, real-time serving, drift-aware retraining.

Principled trade-offs

Explainable GBMs over flashy deep nets; managed vs self-managed chosen on merit; metrics tied to business cost.

Standards across teams

One agent + RAG pattern, one release & monitoring discipline, reused across two clouds — influence beyond a squad.

Bringing people along

I set the patterns, ran the design reviews, and mentored the engineers who built within them.

Exactly the research-meets-production seam, at scale, for a mission that matters — what I want to do next.